DEEP LEARNING - TRANSFER LEARNING - PLANTVILLAGE

Apple Leaf Disease Classification

The complete guide book for the project: the dataset, the exact split, the augmentation that balanced it, seven transfer-learning models, the verified 1-epoch results, and how to run the code yourself.

3,171	12,000	7	100%
ORIGINAL IMAGES	TRAIN IMAGES AFTER AUGMENTATION	MODELS COMPARED	BEST 1-EPOCH TEST ACCURACY



Apple Scab



Black Rot



Cedar Apple Rust



Healthy

Sample images shown above are real images from the project's own test set, one per class.

What this book is for

Anyone who opens the project repository should be able to understand, from this book alone, what data was used, why every step was done, what results have already been verified, and how to continue the training. Nothing in the method is left as a guess: the split, the augmentation counts and the training settings are all written down exactly as they were used.

> **The one rule that protects the whole project:** use the augmented dataset exactly as provided. Do not re-split it and do not re-augment it. Every result in this book is based on that exact dataset.

Contents and how to use this book

1. The problem and the four classes	3
2. Dataset source and original counts	4
3. The split: 80% train, 10% validation, 10% test	5
4. Class imbalance - and the balanced training set	6
5. Augmentation in detail: the cycle method	7
6. Techniques used, techniques not used, preprocessing	8
7. Seven models and partial fine-tuning	9
8. Training settings, no early stopping, reproducibility	10
9. The training plan: 1 epoch, 5 epochs, 30 epochs, fusion	11
10. Evaluation: how the results are judged	12
11. Verified 1-epoch results - all seven models	13
12. Reading the per-class results	14
13. Run the code yourself - GPU or CPU	15
14. Repository guide and pre-training checklist	16

How to use this book

Reading it for the first time: read Chapters 1 to 6 in order. They explain the data, and in this project the data decisions matter more than the model choice - the same seven models behave very differently on an imbalanced set than on the balanced set built here.

Running the code: go straight to Chapter 13. It gives the full order of runs, from the 1-epoch quick test to the 30-epoch final training, and states the only value you ever need to change.

Checking a result: Chapters 11 and 12 hold the verified 1-epoch results, taken from the saved result files of the actual run - test accuracy, validation accuracy, training loss, the confusion matrices and the per-class scores.

> **A note on what is verified and what is planned.** The 1-epoch results in Chapter 11 are complete and verified: all seven models were trained start to end for one epoch and tested. The 5-epoch comparison, the 30-epoch final runs and the feature fusion are the planned next stages, described in Chapter 9. This book does not present planned results as if they had already happened.

1. The problem and the four classes

The task is image classification with four classes. Given a photograph of an apple leaf, the model must say which one of the four it is: Apple Scab, Black Rot, Cedar Apple Rust, or Healthy. There is exactly one correct answer per image, so the model outputs four scores and the highest score is its prediction.

This matters outside the classroom because leaf disease is caught earliest on the leaf itself. A grower who can identify a disease from a leaf photograph can act before it spreads through an orchard, and a model that confuses one disease with another leads to the wrong treatment.



Apple Scab



Black Rot



Cedar Apple Rust



Healthy

Class	What the model has to see
Apple Scab	Dark, velvety spots and patches on the leaf surface.
Black Rot	Dark brown to black lesions, often with a defined edge.
Cedar Apple Rust	Distinctive orange rust spots - the colour itself is a strong signal.
Healthy	A clean green leaf with no disease marks.

Why colour and spots must be protected

The signal in this dataset is the disease mark: its colour, its shape and where it sits on the leaf. That single fact decides several later choices in this book. Augmentation keeps colour changes small, because a large colour change would turn the orange of Cedar Apple Rust into something else. Techniques that erase or mix image regions are not used, because they can hide the very spot the label depends on. Chapter 6 lists those decisions in full.

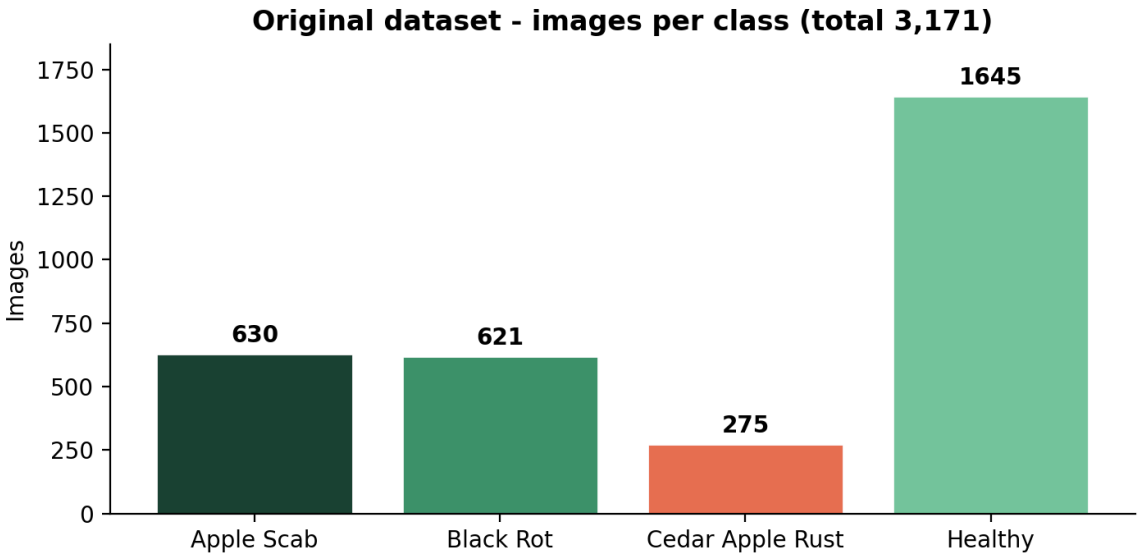
> **One class, one label.** Every image in the dataset belongs to exactly one class folder. The folder name is the label, which is why the folder structure in Chapter 14 must be kept exactly as it is.

2. Dataset source and original counts

The dataset is the **PlantVillage** dataset, and only its Apple subset is used. It was downloaded from the official PlantVillage repository:

```
https://github.com/spMohanty/PlantVillage-Dataset
```

The exact folders used are the colour images under `raw/color/`: `Apple___Apple_scab`, `Apple___Black_rot`, `Apple___Cedar_apple_rust` and `Apple___healthy`. The original images are 256 x 256 pixel RGB colour images, and the downloaded Apple subset is about 51 MB.



Class	Images	Share of dataset
Apple Scab	630	19.9%
Black Rot	621	19.6%
Cedar Apple Rust	275	8.7%
Healthy	1,645	51.9%
Total	**3,171**	**100%**

The table and the chart make the central data problem visible at a glance: more than half of the dataset is a single class, Healthy, while Cedar Apple Rust - a real disease the model must recognise - has only 275 images. Chapter 4 explains what that imbalance does to training, and Chapter 5 explains how it was fixed.

> **The repository carries its own dataset.** The exact dataset used for training - the split and augmented version described in Chapters 3 to 5 - is included with the project, so the work can be reproduced without downloading anything again. The link above is still given in full, because a reader should always be able to check where the data originally came from.

3. The split: 80% train, 10% validation, 10% test

Before any training, the 3,171 original images were divided into three sets. Each set has a different job, and keeping those jobs separate is what makes the final test accuracy believable.

Set	Images	Its job
Train	2,537	The only set the model learns from. This is also the only set that is augmented later.
Validation	317	Checked after every epoch. It shows whether the model is improving and decides which epoch's weights are saved as the best model. It is never trained on.
Test	317	Used once, at the very end, after training is finished. This is the honest final score.

How the split was made

- **Ratio:** 80% train, 10% validation, 10% test.
- **Stratified:** the split was done one class at a time, so every class keeps the same proportions in all three sets. Cedar Apple Rust is 8.7% of the original data and stays about 8.8% of each set.
- **Seed 42:** the random shuffle used seed 42, so running the split again produces exactly the same sets.
- **Rounding:** where 10% of a class is not a whole number of images, it is rounded to the nearest whole image and the training set takes the remainder. That is why Cedar Apple Rust has 219 training images instead of 220, and Healthy has 1,317 instead of 1,316.

Class	Total	Train (original)	Validation	Test
Apple Scab	630	504	63	63
Black Rot	621	497	62	62
Cedar Apple Rust	275	219	28	28
Healthy	1,645	1,317	164	164
Total	**3,171**	**2,537**	**317**	**317**

> **Do not re-split.** The split in this table is the split every result in this book is based on. Splitting again - even with the same ratio - would move images between train and test, and a model could then be tested on images it had already trained on. That inflates the score and makes it meaningless.

4. Class imbalance - and the balanced training set

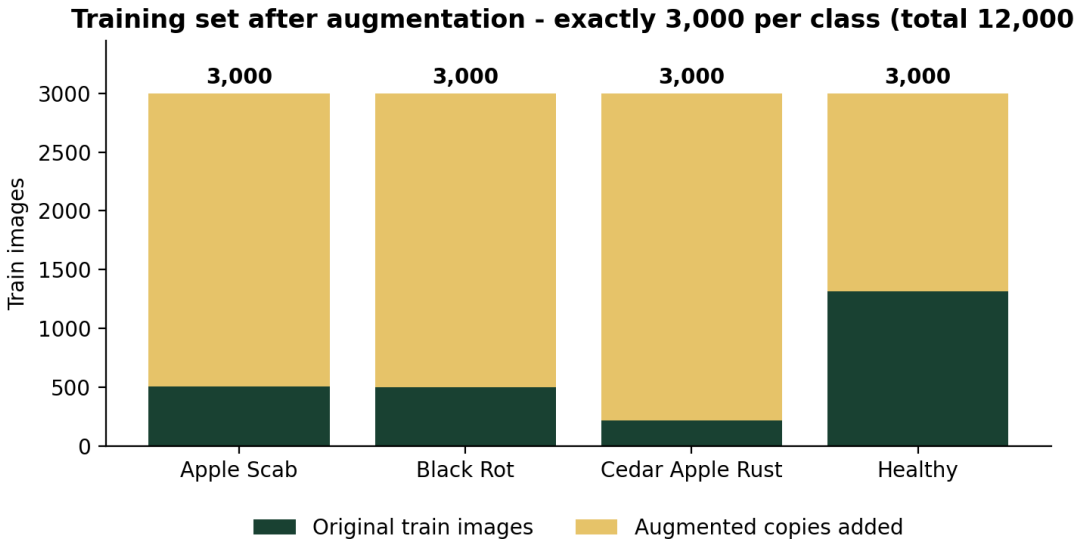
Healthy has 1,645 images. Cedar Apple Rust has 275. Healthy is about **six times** larger than the smallest class, and that gap is a training problem, not just a statistic.

What imbalance does to a model

A model learns most from the class it sees most. Trained on the raw split, it would see a Healthy leaf in more than half of all training steps and a Cedar Apple Rust leaf in fewer than one in eleven. It would get very good at saying "Healthy" and noticeably worse at the smallest disease class - while the overall accuracy could still look high, because the largest class dominates that number too. A model that fails exactly on the rarest disease is not a good disease classifier, whatever its headline accuracy says.

The fix used in this project

The training set - and only the training set - was augmented until every class reached exactly **3,000 images**. Validation and test were never augmented: they stay as original photographs, so the scores measured on them describe performance on real, unseen images.



Split	Scab	Black Rot	Cedar Rust	Healthy	Total
Train (after augmentation)	3,000	3,000	3,000	3,000	**12,000**
Validation (originals only)	63	62	28	164	317
Test (originals only)	63	62	28	164	317
Total images in the project dataset					**12,634**

All 12,634 images were checked: none corrupt, none empty, and every one opens correctly. A visual sample check per class also confirmed that disease spots remain clearly visible in the augmented copies. The next chapter shows exactly how each class reached 3,000.

5. Augmentation in detail: the cycle method

Augmentation makes new training copies from the original training images, each copy changed a little - flipped, rotated, brightened - so the model learns the disease rather than memorising one exact photograph. The method used here is deliberately simple and exactly countable, so anyone can verify the numbers.

The rule

```
needed copies = 3,000 - original training images in that class
```

Augmentation works through the original images **in order**. In one cycle, every original image gets exactly one new augmented copy, and each copy uses a different random combination of the techniques in Chapter 6, so copies are not identical to each other. When a cycle finishes, the next cycle starts again from the first image. The process stops the moment the class reaches exactly 3,000 - the last cycle may be partial.

Worked examples - every class

Class	Original	Copies needed	How the cycles add up	Final
Cedar Apple Rust	219	2,781	12 full cycles = 2,628, then the first 153 originals get a 13th copy. 153 x 13 + 66 x 12 = 2,781.	3,000
Apple Scab	504	2,496	4 full cycles = 2,016, then 480 originals get a 5th copy.	3,000
Black Rot	497	2,503	5 full cycles = 2,485, then 18 originals get a 6th copy.	3,000
Healthy	1,317	1,683	1 full cycle = 1,317, then 366 originals get a 2nd copy.	3,000

Notice the pattern: the smallest class needs the most cycles, and the largest class needs the fewest. Cedar Apple Rust, with only 219 originals, is cycled nearly thirteen times - which is also why its augmented copies must each be genuinely different, not the same change repeated.

Naming and traceability

Every augmented file is named `<original_name>_aug_<cycle>.JPG`. An original and an augmented copy can therefore always be told apart from the file name alone, and any single training image can be traced back to the original photograph it came from. The script that produced the training set is `augment.py`, run with seed 42.

> **Do not re-augment.** The augmented training set is already built and balanced at 3,000 per class. Running augmentation again on top of it would double-count the same originals and destroy the exact counts this book records.

6. Techniques used, techniques not used, preprocessing

Applied to every augmented copy - a different random combination each time

Technique	Range used	Why it helps
Horizontal and vertical flip	Random, on or off	A leaf can be photographed from any side; the disease does not change.
Rotation	-25 to +25 degrees	Leaves are rarely photographed perfectly straight.
Zoom	0.8 to 1.2, centre crop / resize	The camera is never at exactly the same distance.
Brightness	0.8 to 1.2	Light changes between orchard, shade and indoor photographs.
Contrast	0.8 to 1.2	Different cameras and light produce different contrast.
Colour jitter	0.85 to 1.15, kept small on purpose	A slight natural variation - small, so the disease colour is never changed into a different colour.
Gaussian noise	Slight, standard deviation 5	Real photographs carry sensor noise; the model should not be surprised by it.

Deliberately not used

Technique	Reason it was rejected
Cutout / Random Erasing	It deletes a patch of the image - and that patch could be the disease spot itself, leaving a disease label on an image that no longer shows the disease.
Mixup / CutMix	These blend two images together. Blending two different diseases confuses the label: the image would show both, but carry only one name.
Large colour change	Colour is part of the diagnosis here, above all the orange of Cedar Apple Rust. Changing it strongly would train the model on colours that never occur in reality.

Preprocessing - applied to every image, in every set

- **Resize to 224 x 224 pixels.** This is the input size the pretrained models expect.
- **Convert to tensor** and **normalise with the ImageNet values**: mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225]. The pretrained weights were trained with exactly these values, so the leaf images must be prepared the same way for the transferred knowledge to apply.

7. Seven models and partial fine-tuning

No model in this project is trained from scratch. Each one starts from weights pretrained on ImageNet - a very large general image dataset - and is then adapted to apple leaves. That is transfer learning: the model already knows edges, shapes, textures and colour patterns before it sees a single leaf, so good results are possible even in the first epoch, as Chapter 11 shows.

The seven models, in training order

#	Model	Family in one line
1	**ResNet18**	Small residual network - fast, and the natural first baseline.
2	**ResNet50**	Deeper residual network - more capacity than ResNet18.
3	**EfficientNet-B0**	Small EfficientNet - designed for strong accuracy at small size.
4	**DenseNet121**	Dense connections - every layer receives the earlier feature maps.
5	**MobileNetV2**	Lightweight network designed for mobile and low-power devices.
6	**VGG16**	Classic deep network of simple stacked convolutions - large and slow.
7	**EfficientNet-B3**	Larger EfficientNet - more capacity than B0.

Models are trained **one by one**, in this order, never in parallel. Each model finishes, saves its results and its best weights, and only then does the next one start.

Partial fine-tuning - not just the classification head

Only replacing the final classification layer would leave the model's leaf knowledge shallow. This project trains **two parts**: the classification head *and* the last feature block before it, which learns the high-level patterns closest to the final decision. The earlier layers stay frozen, because what they learned from ImageNet - general shapes, edges and textures - is already useful for leaves.

Model	Trained	Frozen
ResNet18 / ResNet50	layer4 + fc	conv1, layer1, layer2, layer3
EfficientNet-B0 / B3	features.7, features.8 + classifier	All earlier feature blocks
DenseNet121	denseblock4 + classifier	All earlier dense blocks
MobileNetV2	features.18 + classifier	All earlier feature blocks
VGG16	features 28-30 + classifier	All earlier feature layers

In every model the original final layer is replaced with a new layer that has **four outputs**, one per class.

8. Training settings, no early stopping, reproducibility

The settings - identical for every model

Setting	Value
Optimizer	Adam
Learning rate	0.001
Loss function	Cross-Entropy
Batch size	32 - so each epoch over 12,000 training images is 375 steps
Input size	224 x 224, ImageNet normalisation (Chapter 6)
Seeds	Python 42, NumPy 42, PyTorch 42

Keeping the settings identical is what makes the comparison fair. If each model had a different learning rate or batch size, a difference in accuracy could come from the settings instead of the model.

No early stopping - a fixed rule of this project

> **Early stopping is not used.** Every run trains for all of its epochs, in full. Validation accuracy is monitored after each epoch and the best-validation weights are saved, but a weak epoch never stops the run early.

The reason is simple: a model's validation score does not improve in a straight line. Stopping at the first dip can throw away a model that would have recovered and finished stronger, and it makes runs hard to compare, because each model would have trained for a different number of epochs. Here, the epoch count is a decision made before the run - 1, 5 or 30 - and the run completes it.

Reproducibility - what to expect when you re-run

- **Same dataset:** the exact split and augmented dataset used for training is provided with the project, so a re-run starts from identical data.
- **Same seeds:** 42 everywhere, as listed above.
- **Close, not bit-identical, on retraining:** retraining can differ by about 1-2% in accuracy because of hardware and floating-point differences. That is normal and is not an error.
- **Exactly identical test results:** use the saved best-model weights and run them on the test set without retraining. The weights are saved after every model, in its results folder, alongside its result file.

> **What is saved per model:** a result file with the training loss, validation accuracy, test accuracy, per-class precision, recall and F1-score, and the confusion matrix - plus the best weights file. Chapter 11's results come directly from those saved files.

9. The training plan: 1 epoch, 5 epochs, 30 epochs, fusion

Training is staged, cheapest first. Each stage answers a different question, and no stage is skipped, because a problem found in a cheap stage saves the cost of the expensive ones.

Stage	Epochs	Question it answers	Status
1. Quick test	1	Does the whole code run start to end, for every model, with no error - and does each model learn at all?	**Complete.** Results in Chapter 11.
2. Comparison	5	Which models are strongest when all are given the same fair run?	Next stage
3. Final training	30	How far do the best models from Stage 2 go with full training?	After Stage 2
4. Feature fusion	-	Does combining the best models beat the best single model?	After Stage 3

Stage 3 - only the best models go forward

Thirty epochs on CPU is slow, and even on a GPU it is the expensive stage. It is therefore run only for the models that earned it in the 5-epoch comparison - not automatically for all seven. Which models qualify is decided by the Stage 2 results together: test accuracy, balanced per-class scores, and a natural train-versus-validation gap (Chapter 10).

Stage 4 - feature fusion, the novel approach

Feature fusion takes the learned feature representations of the best models and combines them into one final model, instead of simply picking a single winner. Only the best models are fused - fusing weak models would import their mistakes. The fused model is then evaluated with exactly the same measures as every individual model, on the same test set, so the comparison stays honest: fusion is kept only if it actually beats the best single model.

What "a natural final model" means

- High test accuracy - and high scores in *every* class, not just the large ones.
- No overfitting: training accuracy is not far above validation accuracy.
- No underfitting: training and validation are not both low.
- Its confusion matrix shows no systematic confusion between two particular classes.

> **Changing stages is changing one number.** Moving from one stage to the next does not mean new code. It means changing the `EPOCHS` value in the same script - 1, then 5, then 30. Chapter 13 shows exactly where.

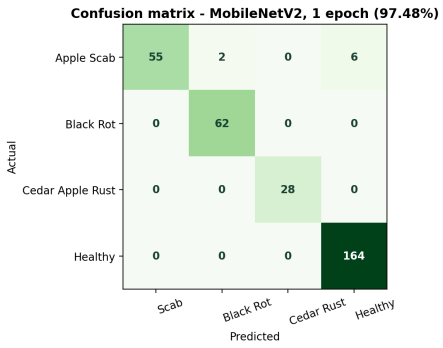
10. Evaluation: how the results are judged

Every model - and later the fused model - is judged with the same set of measures, on the same 317-image test set. Accuracy is only the first of them.

Measure	What it tells you
Test accuracy	The overall share of test images classified correctly. Useful as a headline, misleading on its own.
Per-class precision	Of the images the model called this class, how many really were this class.
Per-class recall	Of the images that really are this class, how many the model found.
Per-class F1-score	Precision and recall combined into one score per class - the quickest way to spot a class the model is weak on.
Confusion matrix	A grid of actual against predicted. It shows *which* classes are confused with each other, which no single number can show.
Training graphs	Training loss and validation accuracy for every epoch - the shape of learning, and the first place overfitting appears.
Bar graph comparison	All models side by side on test accuracy, as in Chapter 11.
Train vs validation gap	A large gap means overfitting; both scores low means underfitting. A good final model has neither.

How to read a confusion matrix

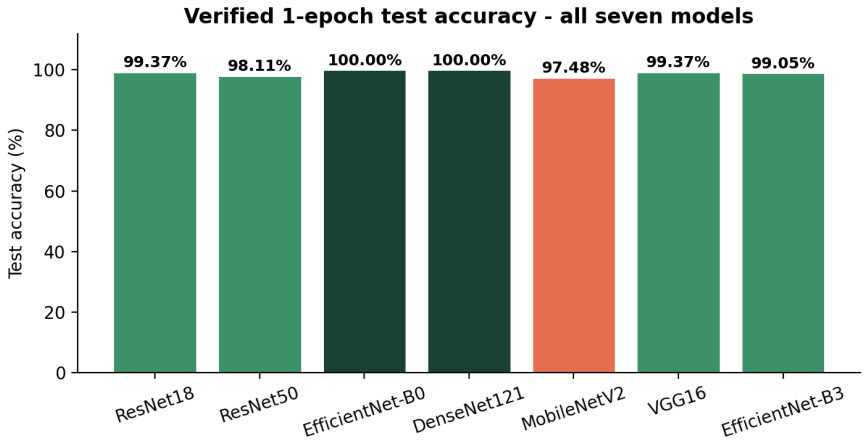
Rows are the actual class, columns are the predicted class. The diagonal is the correct answers. Any number off the diagonal is a specific mistake: its row is the true disease, its column is what the model said instead.



MobileNetV2 is the most instructive example from the 1-epoch run. It classified all 164 Healthy, all 62 Black Rot and all 28 Cedar Apple Rust test images correctly. All eight of its mistakes came from one class: of the 63 Apple Scab images, 55 were correct, 2 were predicted as Black Rot and 6 were predicted as Healthy. One number, 97.48%, hides that pattern; the matrix reveals it in seconds, and it points exactly at what longer training needs to fix.

11. Verified 1-epoch results - all seven models

Each model below was trained start to end for exactly **one epoch** on the full 12,000-image balanced training set, with the settings in Chapter 8, then evaluated once on the 317-image test set. The figures are taken from the saved result file of each model - they are the actual run's output, not estimates.



Model	Train loss	Validation acc.	Test acc.	Test errors / 317
EfficientNet-B0	0.0756	100.00%	**100.00%**	0
DenseNet121	0.0389	99.68%	**100.00%**	0
ResNet18	0.0586	100.00%	99.37%	2
VGG16	0.2738	98.42%	99.37%	2
EfficientNet-B3	0.0852	99.05%	99.05%	3
ResNet50	0.0659	98.74%	98.11%	6
MobileNetV2	0.0908	99.37%	97.48%	8

What these results do - and do not - prove

They prove the pipeline is correct: the data loads, the augmentation has not destroyed the disease signal, every model trains, saves and evaluates without error, and transfer learning is effective on this dataset - even the weakest model is above 97% after a single pass through the data.

They do **not** settle the final ranking. One epoch is a smoke test, not a contest: the order can change at 5 and again at 30 epochs. That is why the staged plan in Chapter 9 exists, and why no final model is declared in this book.

> **Two perfect scores, read carefully.** EfficientNet-B0 and DenseNet121 each classified all 317 test images correctly after one epoch - a strong result on unseen photographs, but still on 317 images, of which only 28 are Cedar Apple Rust. The 5-epoch and 30-epoch stages are what confirm whether it is stable.

12. Reading the per-class results

The table below gives the 1-epoch test recall per class - of the test images that truly belong to a class, how many the model found. Recall is shown because a missed disease is the costliest error in this project, and because the class sizes differ so much (164 Healthy test images against 28 Cedar Apple Rust).

Model	Scab /63	Black Rot /62	Cedar Rust /28	Healthy /164
EfficientNet-B0	63	62	28	164
DenseNet121	63	62	28	164
ResNet18	62	61	28	164
VGG16	62	62	27	164
EfficientNet-B3	61	61	28	164
ResNet50	60	59	28	164
MobileNetV2	55	62	28	164

Three patterns worth noticing

Healthy is never missed. Every model classified all 164 Healthy test images correctly. No model is wasting its errors on the largest class.

Cedar Apple Rust holds up despite being smallest. Six of the seven models found all 28 Cedar Rust images; only VGG16 missed one. This is the clearest early evidence that balancing the training set worked - the class with just 219 original training images is not the class the models fail on.

Apple Scab is where the remaining errors live. Every model's mistakes involve Scab, either missing Scab images or - in the case of ResNet18 and ResNet50 - predicting Cedar Rust for a few images of other classes. Scab's dark patches can resemble other marks at a glance, and this is the boundary that longer training should sharpen.

Precision, recall and F1 in the saved files

The full classification report for every model - precision, recall and F1-score per class, plus macro and weighted averages - is saved in that model's result file next to its weights. For the two perfect models every value is 1.00 in every class. For MobileNetV2, the weakest model, the Scab F1-score of 0.93 against a Cedar Rust F1 of 1.00 states in one line what the matrix on page 12 shows in full.

13. Run the code yourself - GPU or CPU

The code is tested and correct. Run it yourself, on a GPU or on a CPU, with the same augmented dataset provided with the project. Everything else in this book stays exactly as described.

> **Use the `data/` folder exactly as given.** Do not re-split it and do not re-augment it.

Step 1 - Install what the code needs

```
pip install torch torchvision scikit-learn pillow numpy
```

Step 2 - Quick test first: 1 epoch

```
python train_models_1epoch.py
```

This trains every model for one epoch only and saves results in `model_results_1epoch/`. Run it first on any new machine.

Step 3 - Full comparison: 5 epochs

```
python train_models.py
```

This trains the top seven models one by one, five epochs each, in the order listed in Chapter 7. Results and best weights are saved after every model, so an interrupted run never loses a finished model.

Step 4 - Final training: 30 epochs for the best models

Anyone running the code later changes **only the number of epochs** - nothing else:

```
# in train_models.py - change this one number to 30
EPOCHS = 5
```

Dataset, learning rate (0.001), batch size (32) and every other setting stay the same. No early stopping: all epochs run fully.

GPU or CPU - the same code

Machine	What changes
GPU - for example Google Colab or Kaggle GPU	Nothing in the code. It runs as it is, and much faster.
CPU	Nothing in the code. It runs as it is, only slower - and VGG16 is the slowest model.

14. Repository guide and pre-training checklist

What is in the repository

Item	What it is
`data/train/`	12,000 images - 3,000 per class, originals plus augmented copies, in four class folders.
`data/validation/`	317 original images - never augmented.
`data/test/`	317 original images - never augmented, used once at the end of training.
`data/split_report.csv`	The split record - per-class counts for every set.
`train.py`	Single-model training script (ResNet18), 5 epochs.
`train_models.py`	All seven models, one by one, 5 epochs each. Change only `EPOCHS` for the 30-epoch stage.
`train_models_1epoch.py`	The same seven-model script at 1 epoch - the quick code test.
`augment.py`	The augmentation script that produced the balanced training set. Kept for the record - do not re-run it.
`model_results_1epoch/`	The verified 1-epoch results: one result file and one best-weights file per model.
`README.md` + this guide book	The project record and its full explanation, including the verified results.

Inside each of the three data folders are the same four class folders: `Apple_scab`, `Black_rot`, `Cedar_apple_rust` and `Healthy`.

Pre-training checklist

1. The `data/` folder is the provided one - train counts read 3,000 per class, validation 317, test 317.
2. No re-split and no re-augmentation has been done.
3. Dependencies are installed (Chapter 13, Step 1).
4. The 1-epoch script has run clean on this machine before any long run.
5. `EPOCHS` is set to the stage intended - 1, 5 or 30 - and nothing else in the script has been changed.
6. There is enough free space for the saved best weights of every model.

> **The project in one sentence:** a balanced 12,000-image training set built by countable cycle augmentation from a stratified PlantVillage split, seven pretrained models fine-tuned the same fair way, judged per class - and a 1-epoch run in which every model already scores between 97.48% and 100%.